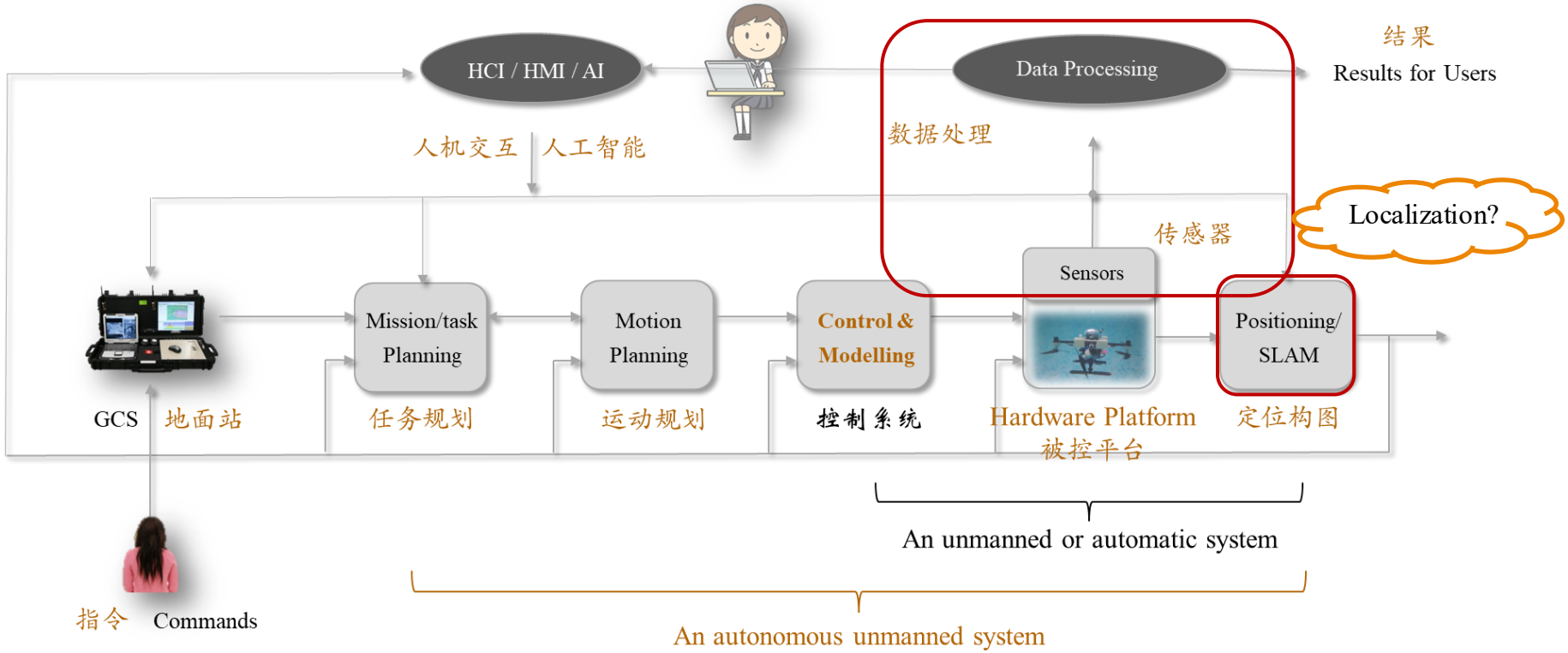


UAV Perception and Positioning

© Copyright Unmanned Systems Research Group



© Copyright Unmanned Systems Research Group



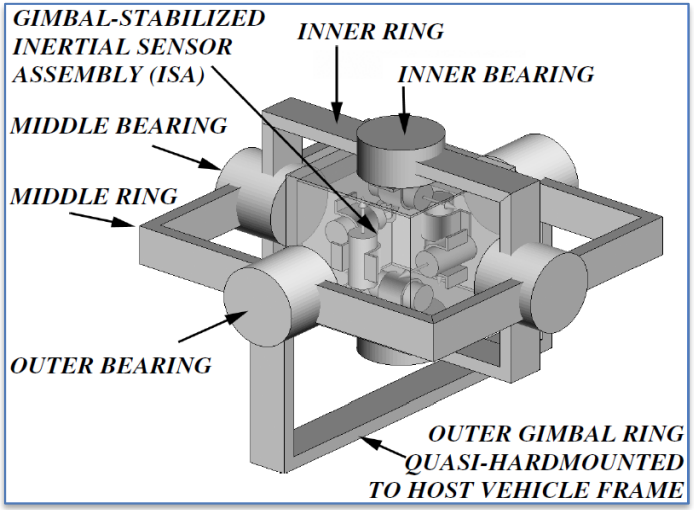
Sensing and Filtering

© Copyright Unmanned Systems Research Group

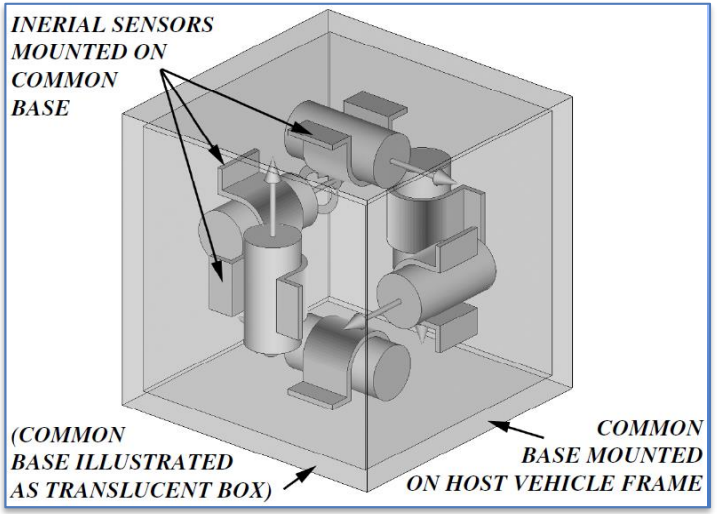
Inertial Measurement Unit

Inertial Measurement Units (IMUs) is a self-contained system that measures linear and angular motion usually with **a triad of gyroscopes** and **triad of accelerometers**. An IMU can either be gimbaled or strapdown, outputting the integrating quantities of angular velocity and acceleration in the sensor/body frame.

Which is more common for UAV? Any disadvantages?



Gimballed IMU

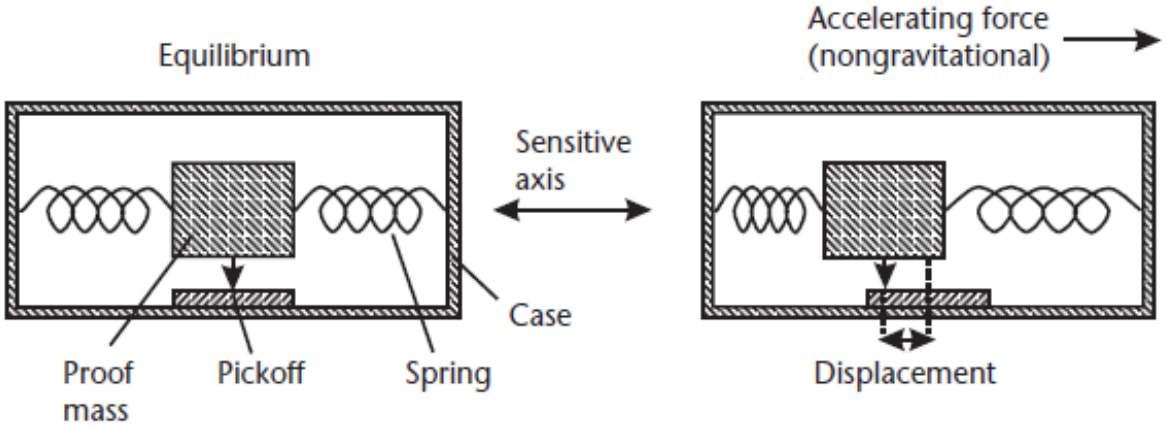


Strapdown IMU

© Copyright Unmanned Systems Research Group

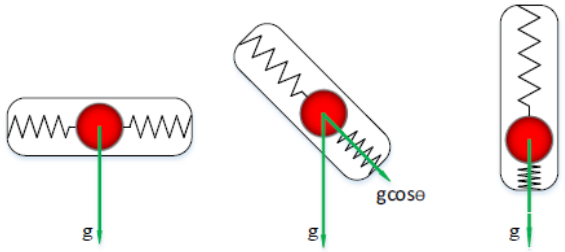
Accelerometer

By attaching a mass to a spring, measuring its deflection, we get a simple accelerometer.



Basic principle: mass inertia effect

- Measure voltage of piezo crystals
- Changes in capacity
- Laser deflection
- Pendulous Integrating (PIGA)



© Copyright Unmanned Systems Research Group

Accelerometer

Define noise model of measurement vector $a^* = (a_x^*, a_y^*, a_z^*)$

$$\underbrace{\vec{a}^*}_{\text{measurement}} = \underbrace{\vec{b}_C + s \cdot M}_{\text{calibration}} \cdot \underbrace{\vec{a} + \vec{b}}_{\text{estimation}} + \underbrace{W\vec{n} + \vec{o}}_{\text{modelling}}$$

Where: a^* = measured acceleration vector

b_c = long term constant bias

s = scale factor

M = misalignement correction matrix

b = short term constant bias

W=crosscorrelation matrix for white noise vector n

o = other (environmental) influences

Simplified model after calibration

$$\mathbf{a}_{\text{nb,IMU}} = \mathbf{a}_{\text{nb}} - \mathbf{b}_a - \mathbf{w}_a,$$

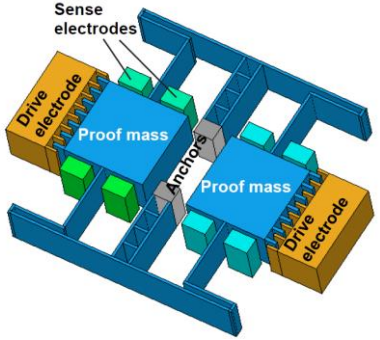
where $\mathbf{b}_a \in R^3$ is unknown but constant measurement biases; and $\mathbf{w}_a \in R^3$ is zero-mean Gaussian white noises.

© Copyright Unmanned Systems Research Group

Rate Gyroscope

Micro Electro-Mechanical System (MEMS) gyroscopes use **Coriolis effect** to indirectly measure angular velocity:

- Proof masses vibrate at frequency ω_r in the 1st axis:
  - Position $p = A\sin(\omega_r t)$
  - Velocity $v = \dot{p} = A\omega_r \cos(\omega_r t)$
- If rotation happens in the 2nd axis, an acceleration $a = -2(v \times \omega)$ will be introduced in the 3rd axis
- This acceleration **can be measured by an accelerometer**





© Copyright Unmanned Systems Research Group

Rate Gyroscope

Simplified model after the calibration

$$\omega^{b}_{b/n,IMU} = \omega^{b}_{b/n} - \mathbf{b}_{\omega} - \mathbf{w}_{\omega},$$

where $\mathbf{b}_{\omega} \in R^3$ is unknown but constant measurement biases; and $\mathbf{w}_{\omega} \in R^3$ is zero-mean Gaussian white noises.

Define noise model of measurement vector $\omega^* = (\omega^*_x, \omega^*_y, \omega^*_z)$

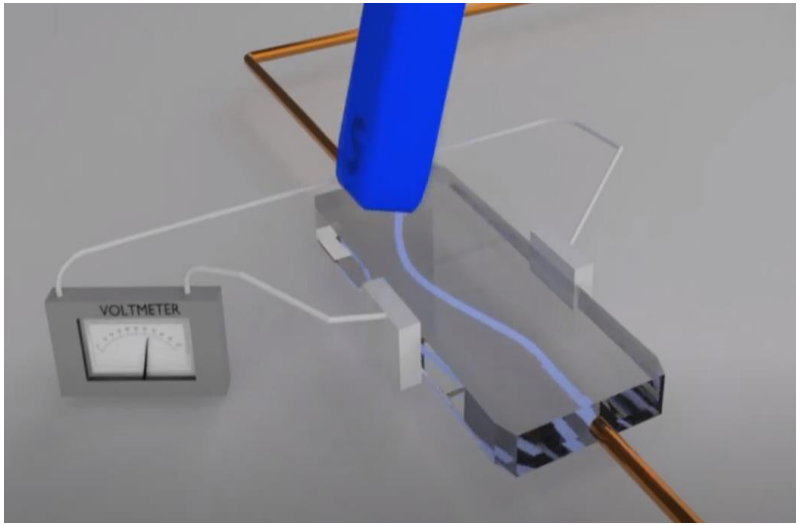
$$\underbrace{\vec{\omega}^*}_{\text{measurement}} = \underbrace{\vec{b}_C + s \cdot M}_{\text{calibration}} \cdot \underbrace{\vec{\omega} + \vec{b}}_{\text{estimation}} + \underbrace{W\vec{n}}_{\text{modelling}} + \vec{o}$$

Where: ω^* = measured angular velocities
b_c = long term constant bias
s = scale factor
M = misalignment correction matrix
b = short term constant bias
W= crosscorrelation matrix for white noise vector n
o = other (environmental) influences

© Copyright Unmanned Systems Research Group

Magnetometer

Detecting changes in a sensor's physical properties (such as voltage, resistance, or quantum state) when exposed to a magnetic field, thereby measuring the field's strength and direction.





© Copyright Unmanned Systems Research Group

Global Positioning System

The Global Positioning System (GPS) is a space-based navigation system that provides **location** and **time** information in all weather conditions, anywhere on or near the Earth where there is an unobstructed line of sight to **four or more GPS satellites**.

- Based on time and the known position of specialized satellites;
- Independent of time of day or weather;
- Begin in 1973 and fully operational since April 1995;
- Currently 30 active satellites (6 paths, 55 deg. inclination);
- Carrier frequency is 1575.42 MHz (1227.6 MHz).

Other Similar Systems

- GLONASS – Russian
- BEIDO – China
- GALILEO – Europe

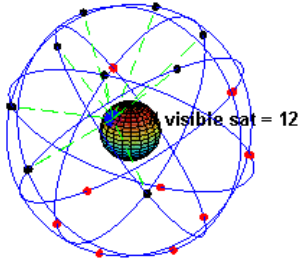
Still remember what is the frequency for RC?

© Copyright Unmanned Systems Research Group

Global Positioning System

Space Satellites:

- NAVSTAR: 30 satellites with atomic clocks
- At least 5 always visible (usually 7-8)
- 20000 km alt. 13000 km/h speed



Control Station:

- Master control for satellites
- Update satellite position and atomic clocks



User Reception:

- GPS receiver with antenna and precise clock (crystal)
- Geoid model for height estimation



© Copyright Unmanned Systems Research Group

Global Positioning System

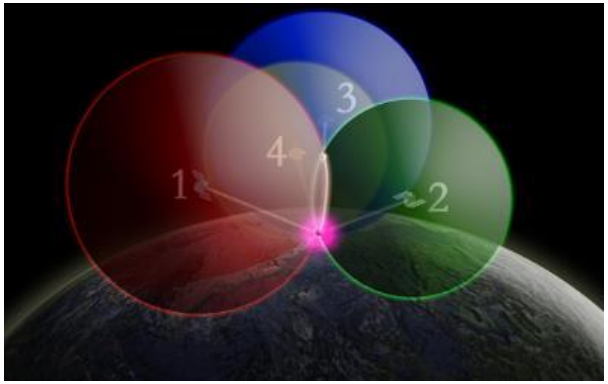
Navigation Equation

The receiver uses messages received from satellites to determine the satellite positions and time sent. For n satellites, the equations to satisfy are:

$$(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2 = \left([\tilde{t}_i - b - s_i]c\right)^2, \; i = 1, 2, \ldots, n$$

where

(x_i, y_i, z_i): position of satellite i;
(x, y, z): receiver position;
c: speed of light;
b: receiver's clock bias;
$\tilde{t}_i$: on-board receiver clock;
s_i: satellite time.



Least Squares Solution

$$\left(\hat{x}, \hat{y}, \hat{z}, \hat{b}\right) = \underset{(x,y,z,b)}{\arg\min} \sum_i \left(\sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} + bc - p_i\right)^2$$

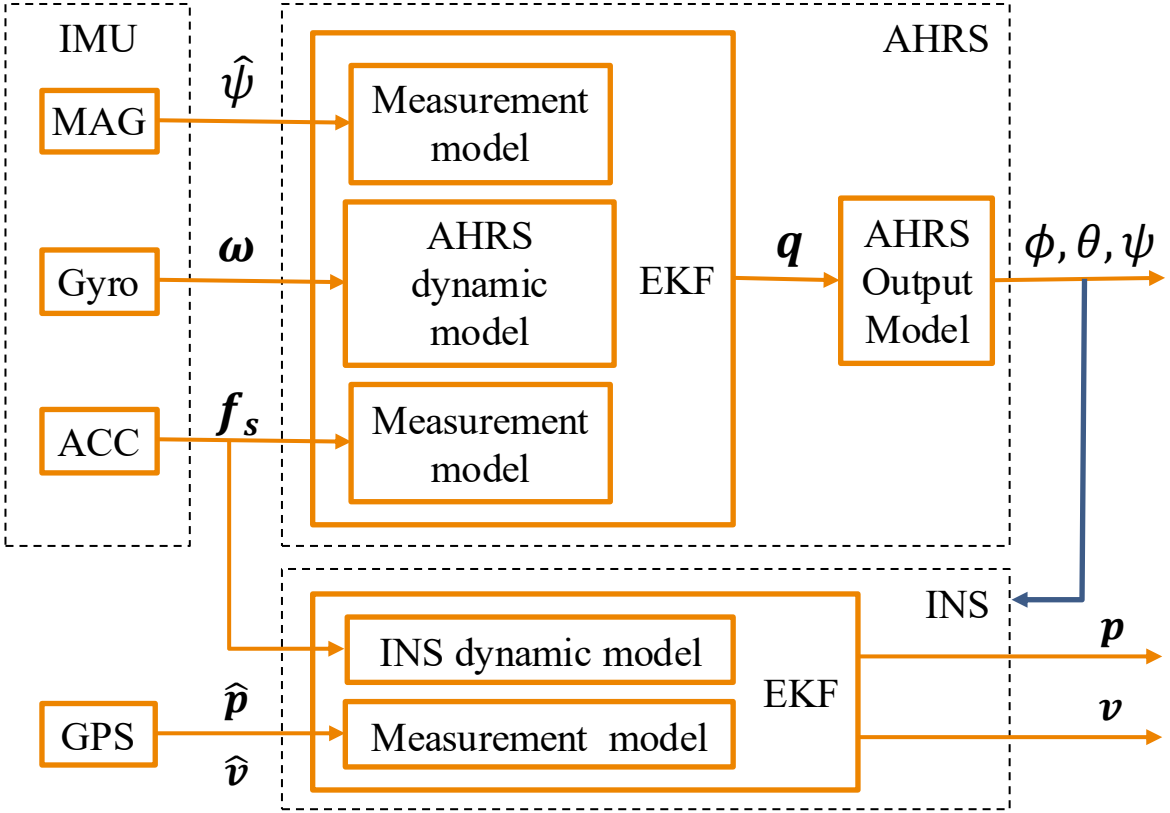
where

$$p_i = \left(\tilde{t}_i - s_i\right) c$$ is called the pseudo-range

For 3D localization, we need at least 4 satellites!

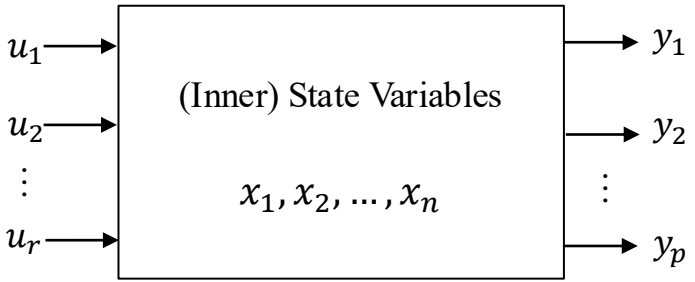
© Copyright Unmanned Systems Research Group

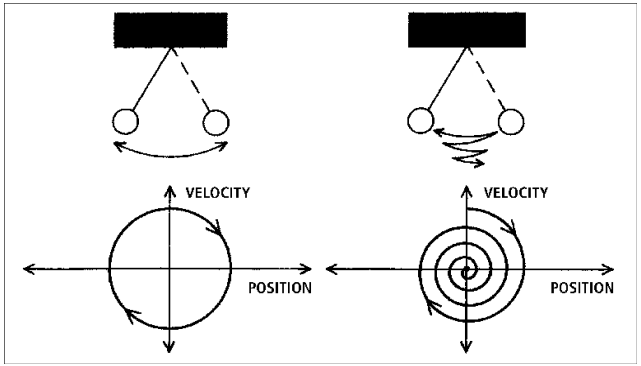
GPS-Aided Inertial Navigation System



© Copyright Unmanned Systems Research Group

State Space Representation of a System





What is the input?

- State: the smallest set of variables (called State Variables or State Vector) such that the knowledge of these variables at $t = t_0$, together with knowledge of the input for $t \geq t_0$, completely determines the behavior and outputs of the system for any time t_0 to t.

- The number of state variables to completely define the dynamics of the system is equal to the number of integrator involved in the system (System Order).

- The above system has r inputs, p outputs and n state variables

© Copyright Unmanned Systems Research Group

State Space Representation of a System

$$\begin{cases} \dot{x}(t) = Fx(t) + Gu(t) & \text{State equation} \\ y(t) = Hx(t) + Ju(t) & \text{Output equation} \end{cases}$$

- $x = State\ Vector$
- $\dot{x} = \frac{dx(t)}{dt} = Derivative\ of\ the\ State\ Vector$
- $u = Input\ Vector$
- $y = Output\ Vector$
- $F = State\ Matrix = System\ Matrix$
- $G = Input\ Matrix$
- $H = Output\ Matrix$
- $J = Feedthrough\ Matrix\ (instantaneous\ influence\ of\ the\ input\ on\ output)$

© Copyright Unmanned Systems Research Group

State Space Representation of a System

$$\begin{cases} \dot{x}(t) = Fx(t) + Gu(t) \\ y(t) = Hx(t) + Ju(t) \end{cases}$$

$C(F,G) = [G \;\; FG \;\; F^2G \;\; \ldots \;\; F^{n-1}G]$ Controllability Matrix

$V(F,H) = [H \;\; HF \;\; HF^2 \ldots \;\; HF^{n-1}]^T$ Observability Matrix

A system is said to be (state) controllable at time t_0, if there exists a finite $t_1 > t_0$ such that for any $x(t_0)$ and any x_1, there exists an input $u_{[t_0,t_1]}$ that will transfer the state $x(t_0)$ to the state x_1 at time t_1. Otherwise, the system is said to be uncontrollable at time t_0.

$\text{rank}(C) = n$

$\det(C) \neq 0$

A system is said to be (completely state) observable at time t_0, if there exists a finite $t_1 > t_0$ such that for any $x(t_0)$ at time t_0, the knowledge of the input $u_{[t_0,t_1]}$ and the output $y_{[t_0,t_1]}$ over the time interval $[t_0, t_1]$ suffices to determine the state x_0. Otherwise, the system is said to be unobservable at time t_0.

$\text{rank}(V) = n$

$\det(V) \neq 0$

© Copyright Unmanned Systems Research Group

Kalman Filter

What is Kalman Filter?

Given the linear dynamical system:

Difference between predicted and measured

$$x(k+1) = F(k)x(k) + G(k)u(k) + v(k)$$
$$y(k) = H(k)x(k) + w(k)$$

Where is Ju(t) item?

$x(k)$ is the n-dimensional state vector (unknown)

$u(k)$ is the m-dimensional input vector (known)

$y(k)$ is the p-dimensional output vector (known, measured)

$F(k), G(k), H(k)$ are appropriately dimensioned system matrices (known)

$v(k), w(k)$ are zero-mean, white Gaussian noise with (known) covariance matrices $Q(k), R(k)$

Kalman Filter is a recursive algorithm that provides the 'best' or 'most probable' estimate of the state vector *x*.

© Copyright Unmanned Systems Research Group

Kalman Filter

Why use Kalman Filter?

$$x(k+1) = F(k)x(k) + G(k)u(k) + v(k)$$
$$y(k) = H(k)x(k) + w(k)$$

- State estimation via probability optimization from both inputs and measurements
- Noise smoothing
- Realtime scalable (computes next estimate using only most recent measurement)

© Copyright Unmanned Systems Research Group

Kalman Filter

How does Kalman Filter work?

$$x(k+1) = F(k)x(k) + G(k)u(k) + v(k)$$
$$y(k) = H(k)x(k) + w(k)$$

1. Prediction based on last estimate:

$$\hat{x}(k+1|k) = F(k)\hat{x}(k|k) + G(k)u(k)$$
$$\hat{y}(k) = H(k)\hat{x}(k+1|k)$$

2. Calculate correction based on prediction and current measurement:

$$\Delta x = f\left(y(k+1), \hat{x}(k+1|k)\right)$$

3. Update prediction: $\hat{x}(k+1|k+1) = \hat{x}(k+1|k) + \Delta x$

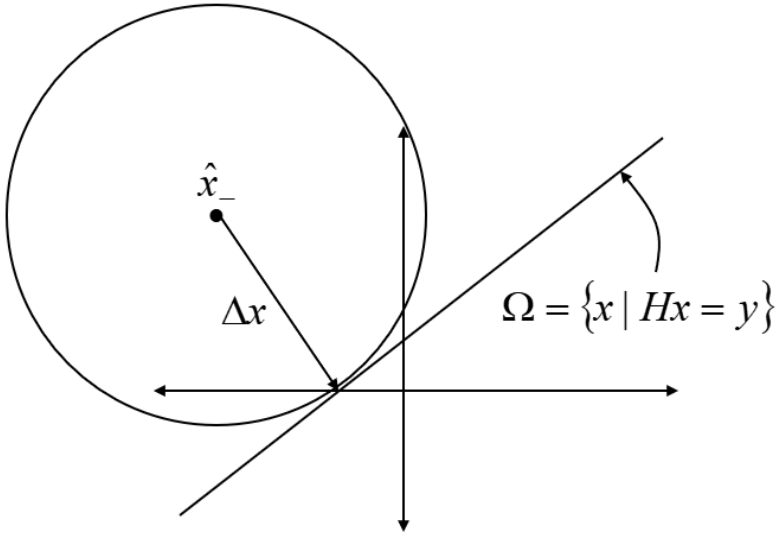
© Copyright Unmanned Systems Research Group

Kalman Filter (A Simple Special Case)

If measurement has zero noise and variance of *x* is the same in all axis ...

$$y = Hx$$

Given prediction $\hat{x}_-$ and output y, find Δx so that $\hat{x} = \hat{x}_- + \Delta x$ is the "best" estimate of x.



$y - H\hat{x}$

$$\Delta x = H^T \left(HH^T\right)^{-1} \nu$$

© Copyright Unmanned Systems Research Group

Kalman Filter (A Simple Special Case)

System:

$$x(k+1) = Fx(k) + Gu(k) + v(k)$$
$$y(k) = Hx(k)$$

1. Prediction:

$$\hat{x}(k+1 \mid k) = F\hat{x}(k \mid k) + Gu(k)$$

2. Compute correction:

$$\Delta x = H^T \left(HH^T\right)^{-1} \left(y(k+1) - H\hat{x}(k+1 \mid k)\right)$$

3. Update:

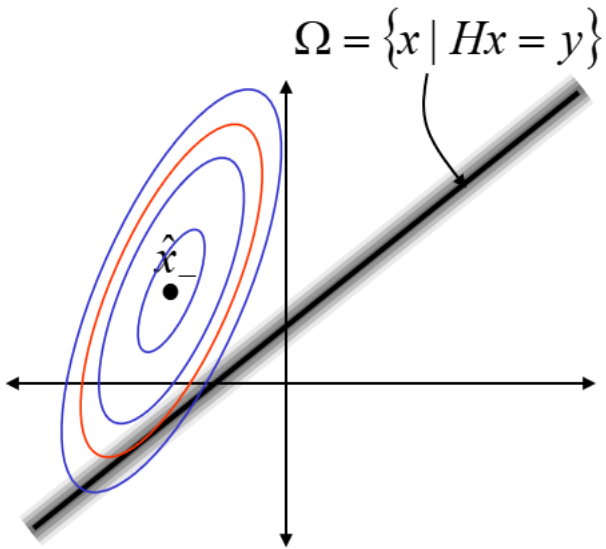
$$\hat{x}(k+1 \mid k+1) = \hat{x}(k+1 \mid k) + \Delta x$$

© Copyright Unmanned Systems Research Group

Kalman Filter (The Ultimate General Case)

If measurement has noises too …

$$y = Hx + w$$



You do not have a hyperplane to directly aim for, you can't solve this with simple algebra!

You must solve an optimization problem.

The answer:

$$\Delta x = PH^T \left(HPH^T + R\right)^{-1} \left(y - H\hat{x}_-\right)$$

Kalman Filter (The Ultimate General Case)

System:

$$x(k+1) = Fx(k) + Gu(k) + v(k)$$
$$y(k) = Hx(k) + w(k)$$

1. Predict

$$\hat{x}(k+1|k) = F\hat{x}(k|k) + Gu(k)$$
$$P(k+1|k) = FP(k|k)F^T + Q$$

2. Correction

$$S = HP(k+1|k)H^T + \mathrm{R}$$
$$W = P(k+1|k)HS^{-1}$$
$$\Delta x = W\left(y(k+1) - H\hat{x}(k+1|k)\right)$$

3. Update

$$\hat{x}(k+1|k+1) = \hat{x}(k+1|k) + W\nu$$
$$P(k+1|k+1) = P(k+1|k) - WSW^T$$

W: Kalman gain (Optimal Weight Matrix)

© Copyright Unmanned Systems Research Group

Extended Kalman Filter (Direct Noise Form)

The extended Kalman filter is used for system state estimation with **nonlinear dynamics** or is usually seen as the **nonlinear version of Kalman filter**. We consider a given system characterized by

$$\mathbf{x}_k = \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{w}_{k-1}$$

and

$$\mathbf{y}_k = \mathbf{h}(\mathbf{x}_k) + \mathbf{v}_k,$$

where $\mathbf{x}_k$ is the true state vector at time k, $\mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1})$ and $\mathbf{h}(\mathbf{x}_k)$ are some nonlinear (or linear) functions, and $\mathbf{w}$ and $\mathbf{v}$ are the process and measurement noises. The noises, $\mathbf{w}$ and $\mathbf{v}$, are assumed to be independent, zero mean, and with normal probability distributions

$$\mathbf{w}_k \sim \mathrm{N}(0, \mathbf{Q}_{\mathrm{cov}})$$

and

$$\mathbf{v}_k \sim \mathrm{N}(0, \mathbf{R}_{\mathrm{cov}}),$$

where $\mathbf{Q}_{\mathrm{cov}}$ and $\mathbf{R}_{\mathrm{cov}}$ are the process and measurement noise covariance matrices.

© Copyright Unmanned Systems Research Group

Extended Kalman Filter (Direct Noise Form)

Same as Kalman Filter, the computation of EKF involves two steps:

1. **Prediction Step**: The prediction step uses the previous time step to produce an estimation of the state at the current time step. In this stage we have the predicted state estimate, and the predicted estimated covariance as follows:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{f}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1})$$

and

$$\mathbf{P}_{k|k-1} = \mathbf{F}_{k-1}\mathbf{P}_{k-1|k-1}\mathbf{F}_{k-1}^{\mathrm{T}} + \mathbf{Q}_{\mathrm{cov}}$$

where $\mathbf{F}_{k-1}$, the Jacobian matrix of the partial derivative of the function $\mathbf{f}$ with respect to $\mathbf{x}$ at time $k-1$, is defined by

$$\mathbf{F}_{k-1} = \left.\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}}$$

© Copyright Unmanned Systems Research Group

Extended Kalman Filter (Direct Noise Form)

2. Correction Step: In the correction step, the current predicted estimate is combined with current observation information to refine the state estimate. At time instance k, we need to compute the Kalman gain $\mathbf{K}_k$ and then update the state estimate and error covariance as follows:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}_k^{\mathrm{T}}(\mathbf{H}_k\mathbf{P}_{k|k-1}\mathbf{H}_k^{\mathrm{T}} + \mathbf{R}_{\mathrm{cov}})^{-1},$$
$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k(\mathbf{y}_k - \mathbf{H}_k\hat{\mathbf{x}}_{k|k-1}),$$

and

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k\mathbf{H}_k)\mathbf{P}_{k|k-1},$$

where $\mathbf{H}_k$, the Jacobian matrix of the partial derivative of the function $\mathbf{h}$ with respect to $\mathbf{x}$ at time k, is defined by

$$\mathbf{H}_k = \left.\frac{\partial \mathbf{h}}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k|k-1}}$$

© Copyright Unmanned Systems Research Group

Extended Kalman Filter (Indirect Noise Form)

$$
\begin{aligned}
\dot{\mathbf{x}} &= \mathbf{f}_{\mathrm{c}}(\mathbf{x}, u, \mathbf{w}) & p(\mathbf{w}) &\sim \mathrm{N}(0, \mathbf{Q}) \\
\mathbf{y} &= \mathbf{h}_{\mathrm{c}}(\mathbf{x}, \mathbf{v}) & p(\mathbf{v}) &\sim \mathrm{N}(0, \mathbf{R})
\end{aligned}
$$

$$
\mathbf{F}_k = \frac{\partial f_c}{\partial x} \qquad \mathbf{W}_k = \frac{\partial f_c}{\partial w} \qquad \mathbf{H}_k = \frac{\partial h_c}{\partial x} \qquad \mathbf{V}_k = \frac{\partial h_c}{\partial v}
$$

- Time update (prediction):

$$
\begin{aligned}
\hat{\mathbf{x}}_{k|k-1} &= \mathbf{f}_{\mathrm{d}}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}, 0) \\
\mathbf{P}_{k|k-1} &= \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}'_k + \mathbf{W}_k \mathbf{Q}_k \mathbf{W}'_k
\end{aligned}
$$

- Measurement update (correction):

$$
\begin{aligned}
\mathbf{K}_k &= \mathbf{P}_{k|k-1} \mathbf{H}'_k (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}'_k + \mathbf{V}_k \mathbf{R}_k \mathbf{V}'_k)^{-1} \\
\hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k [\mathbf{y}_k - \mathbf{h}_{\mathrm{d}}(\hat{\mathbf{x}}_{k|k-1})] \\
\mathbf{P}_{k|k} &= \mathbf{P}_{k|k-1} - \mathbf{K}_k \mathbf{H}_k \mathbf{P}_{k|k-1}
\end{aligned}
$$

© Copyright Unmanned Systems Research Group

EKF: Attitude and Heading Reference System

Euler angle method

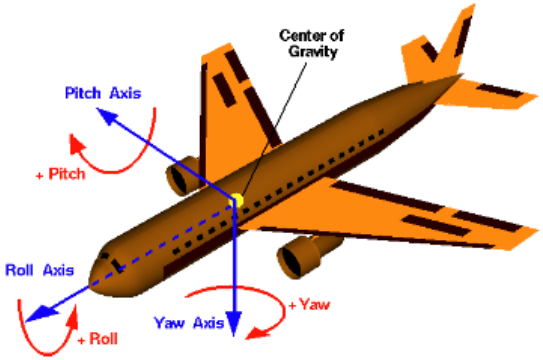
- Sequenced roll, pitch, yaw
- Non-linear computations
- Gimbal lock singularities

$$\sec\theta = \frac{1}{\cos\theta}$$

Angular rate: $\boldsymbol{\omega}$

Roll
Pitch
Yaw

$$\begin{pmatrix}\dot{\phi}\\ \dot{\theta}\\ \dot{\psi}\end{pmatrix} = \underbrace{\begin{bmatrix}1 & \sin\phi\tan\theta & \cos\phi\tan\theta\\ 0 & \cos\phi & -\sin\phi\\ 0 & \sin\phi\sec\theta & \cos\phi\sec\theta\end{bmatrix}}_{\mathbf{H}} \begin{pmatrix}p\\ q\\ r\end{pmatrix}$$



Discretization

$$\boldsymbol{\Phi}(t_k) = \boldsymbol{\Phi}(t_{k-1}) + \int_{t_{k-1}}^{t_k} \mathbf{H}\boldsymbol{\omega}\mathrm{d}t$$

$\mathbf{F}_k, \mathbf{W}_k$?

© Copyright Unmanned Systems Research Group

EKF: Attitude and Heading Reference System

Quaternion method

Complex number

$$\boldsymbol{i}^2 = \boldsymbol{j}^2 = \boldsymbol{k}^2 = \boldsymbol{ijk} = -1 \qquad |\boldsymbol{v}| = 1 \qquad |\boldsymbol{q}| = 1$$

$$\boldsymbol{q} = q_0 + q_1\boldsymbol{i} + q_2\boldsymbol{j} + q_3\boldsymbol{k} = \cos\frac{\theta}{2} + \sin\frac{\theta}{2}(\boldsymbol{v})$$

$$\boldsymbol{q}^{-1} = q_0 - q_1\boldsymbol{i} - q_2\boldsymbol{j} - q_3\boldsymbol{k} = \cos\frac{\theta}{2} - \sin\frac{\theta}{2}(\boldsymbol{v})$$

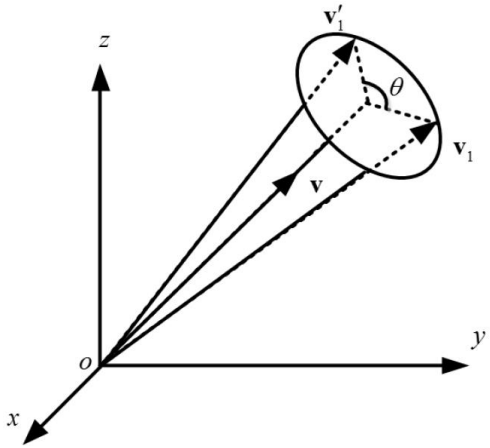
Symbol overload
NOT the same θ

$$q_0 = \cos\frac{\phi}{2}\cos\frac{\theta}{2}\cos\frac{\psi}{2} + \sin\frac{\phi}{2}\sin\frac{\theta}{2}\sin\frac{\psi}{2}$$

$$q_1 = \sin\frac{\phi}{2}\cos\frac{\theta}{2}\cos\frac{\psi}{2} - \cos\frac{\phi}{2}\sin\frac{\theta}{2}\sin\frac{\psi}{2}$$

$$q_2 = \cos\frac{\phi}{2}\sin\frac{\theta}{2}\cos\frac{\psi}{2} + \sin\frac{\phi}{2}\cos\frac{\theta}{2}\sin\frac{\psi}{2}$$

$$q_3 = \cos\frac{\phi}{2}\cos\frac{\theta}{2}\sin\frac{\psi}{2} - \sin\frac{\phi}{2}\sin\frac{\theta}{2}\cos\frac{\psi}{2}$$



$$\begin{bmatrix} 0 \\ x_2 \\ y_2 \\ z_2 \end{bmatrix} = \boldsymbol{q} \cdot \begin{bmatrix} 0 \\ x_1 \\ y_2 \\ z_3 \end{bmatrix} \cdot \boldsymbol{q}^{-1}$$

Rotate a point (x_1, y_1, z_1) around axis $\boldsymbol{v}$ by angle θ and arrive at (x_2, y_2, z_2)

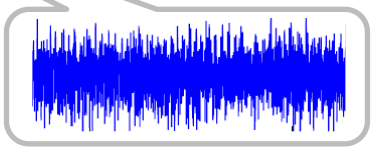
© Copyright Unmanned Systems Research Group

EKF: Attitude and Heading Reference System

Quaternion method

$$\begin{pmatrix} \dot{q}_0 \\ \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{pmatrix} = \frac{1}{2} \begin{bmatrix} 0 & -p & -q & -r \\ p & 0 & r & -q \\ q & -r & 0 & p \\ r & q & -p & 0 \end{bmatrix} \begin{pmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{pmatrix}$$

Input noise



$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{\Omega q}$$

Discretization

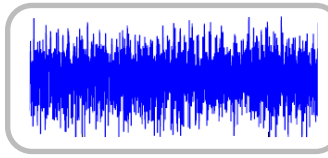
Re-normalization

$$\mathbf{q}(t_k) = \mathbf{q}(t_{k-1}) + \int_{t_{k-1}}^{t_k} \frac{1}{2}\mathbf{\Omega q} \mathrm{d}t \quad \Rightarrow \quad q_0^2 + q_1^2 + q_3^2 + q_4^2 = 1$$

© Copyright Unmanned Systems Research Group

EKF: Attitude and Heading Reference System

Update by accelerometer measurement:



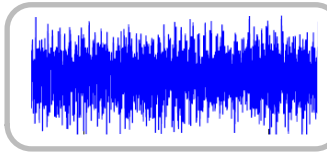
Measurement noise

$$\begin{pmatrix} f_{\mathrm{u}} \\ f_{\mathrm{v}} \\ f_{\mathrm{w}} \end{pmatrix} = \begin{pmatrix} -2g(q_1q_3 - q_0q_2) \\ -2g(q_0q_1 + q_2q_3) \\ -g(q_0^2 - q_1^2 - q_2^2 + q_3^2) \end{pmatrix}$$

$\mathbf{H}_k, \mathbf{V}_k$?

Body acceleration

Update by magnetometer measurement:



Measurement noise

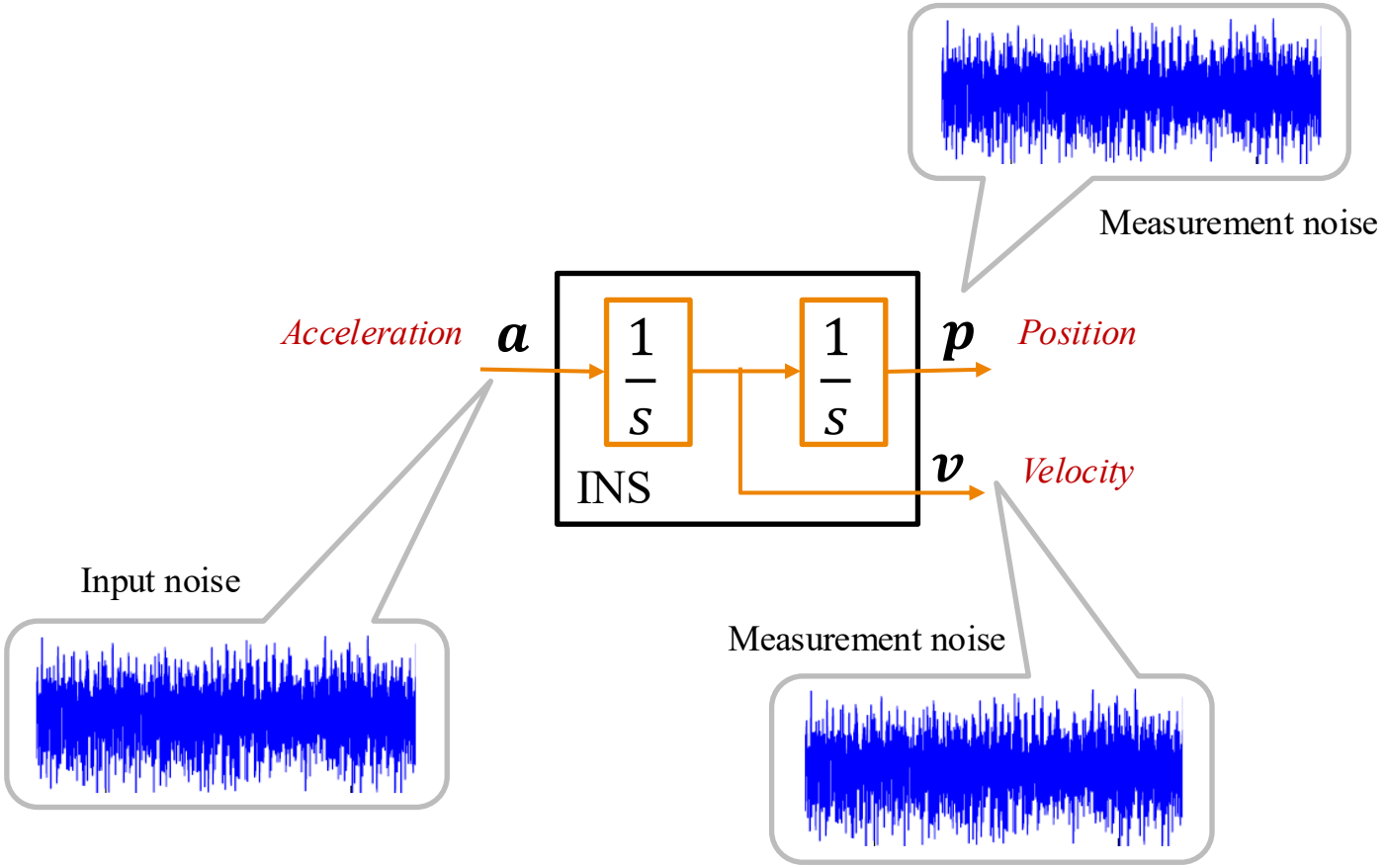
$$\psi = \arctan \frac{2(q_1q_2 + q_0q_3)}{q_0^2 + q_1^2 - q_2^2 - q_3^2}$$

$\mathbf{H}_k, \mathbf{V}_k$?

Heading

© Copyright Unmanned Systems Research Group

EKF: Inertial Navigation System



© Copyright Unmanned Systems Research Group

EKF: Inertial Navigation System

Process Model (works for navigation within small earth surface area):

$$\begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \\ b_x \\ b_y \\ b_z \end{bmatrix}_k = \underbrace{\begin{bmatrix} \begin{bmatrix} 1 & 0 & 0 & dt & 0 & 0 & -dt^2/2 & 0 & 0 \\ 0 & 1 & 0 & 0 & dt & 0 & 0 & -dt^2/2 & 0 \\ 0 & 0 & 1 & 0 & 0 & dt & 0 & 0 & -dt^2/2 \\ 0 & 0 & 0 & 1 & 0 & 0 & -dt & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & -dt & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & -dt \end{bmatrix} \begin{bmatrix} I_6 & 0 \\ 0 & \boldsymbol{R}_{g/b} \end{bmatrix} \\ \hline \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \end{bmatrix}}_{F} \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \\ b_x \\ b_y \\ b_z \end{bmatrix}_{k-1} + \underbrace{\begin{bmatrix} dt^2/2 & 0 & 0 \\ 0 & dt^2/2 & 0 \\ 0 & 0 & dt^2/2 \\ dt & 0 & 0 \\ 0 & dt & 0 \\ 0 & 0 & dt \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{R}_{g/b} & \begin{matrix} 0 \\ 0 \\ g \end{matrix} \end{bmatrix}}_{G} \begin{bmatrix} a_x \\ a_y \\ a_z \\ 1 \end{bmatrix}_k$$

$$Q = G \begin{bmatrix} q_x & 0 & 0 & 0 \\ 0 & q_y & 0 & 0 \\ 0 & 0 & q_z & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} G'$$

Measurement Models:

$$\begin{bmatrix} \mathbf{p} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} \boldsymbol{I} & 0 & 0 \\ 0 & \boldsymbol{I} & 0 \end{bmatrix} \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \\ \mathbf{b} \end{bmatrix} \quad [\mathbf{p}] = [\mathbf{I} \quad 0 \quad 0] \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \\ \mathbf{b} \end{bmatrix}$$

© Copyright Unmanned Systems Research Group

EKF: Inertial Navigation System

Process Model (works for long-range navigation on the earth):

Process Model:

$$\begin{pmatrix} \dot{\varphi} \\ \dot{\lambda} \\ \dot{h} \\ \dot{v}_{\mathrm{x}} \\ \dot{v}_{\mathrm{y}} \\ \dot{v}_{\mathrm{z}} \\ \dot{b}_{\mathrm{x}} \\ \dot{b}_{\mathrm{y}} \\ \dot{b}_{\mathrm{z}} \end{pmatrix} = \begin{pmatrix} \frac{v_{\mathrm{x}}}{M+h} \\ \frac{v_{\mathrm{y}}}{(N+h)\cos\varphi} \\ -v_{\mathrm{z}} \\ -\frac{v_{\mathrm{y}}^2 \sin\varphi}{(N+h)\cos\varphi} + \frac{v_{\mathrm{x}} v_{\mathrm{z}}}{M+h} + f_{\mathrm{x}} - b_{\mathrm{x}} \\ \frac{v_{\mathrm{x}} v_{\mathrm{y}} \sin\varphi}{(N+h)\cos\varphi} + \frac{v_{\mathrm{y}} v_{\mathrm{z}}}{N+h} + f_{\mathrm{y}} - b_{\mathrm{y}} \\ -\frac{v_{\mathrm{y}}^2}{N+h} - \frac{v_{\mathrm{x}}^2}{M+h} + g + f_{\mathrm{z}} - b_{\mathrm{z}} \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

When a gimballed IMU is usually used, global frame acceleration can be measured directly

For a strapdown IMU, b2g rotation need to be applied

Measurement Models:

$$\begin{bmatrix} \mathbf{p} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} \boldsymbol{I} & 0 & 0 \\ 0 & \boldsymbol{I} & 0 \end{bmatrix} \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \\ \mathbf{b} \end{bmatrix} \qquad [\mathbf{p}] = [\mathbf{I} \quad 0 \quad 0] \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \\ \mathbf{b} \end{bmatrix}$$

© Copyright Unmanned Systems Research Group



SLAM:

Simultaneous Localization and Mapping

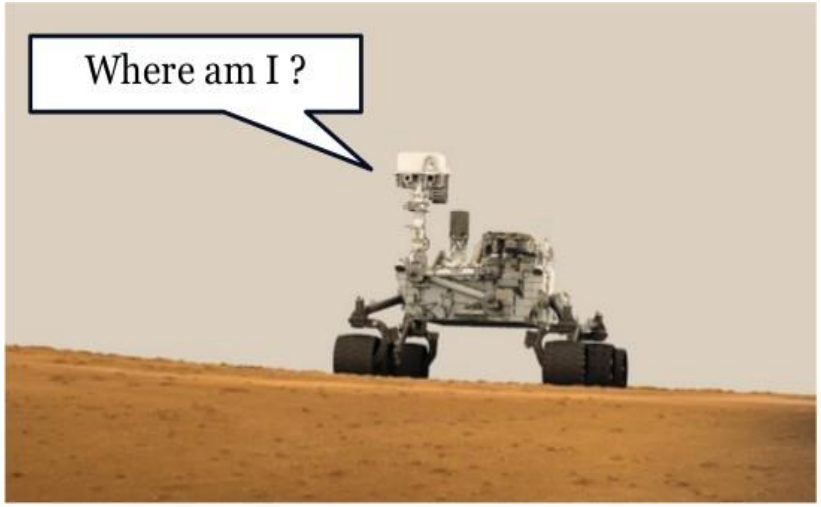
© Copyright Unmanned Systems Research Group

Overview of SLAM

The computational problem of constructing or updating a map of an unknown environment while simultaneously keeping track of an agent's (e.g., UAV) location within it.

Chicken-and-Egg problem:

- ✓ A map is needed for localization
- ✓ A pose estimation is needed for mapping





© Copyright Unmanned Systems Research Group

Application scenarios

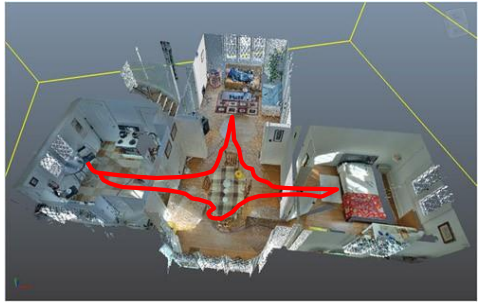
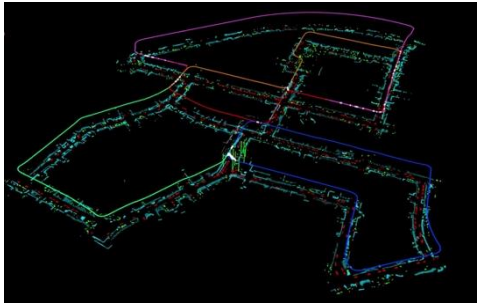
Localization:

- Under the inaccurate GPS
- GPS-denied environment





- Scenarios in which a prior map is not available and needs to be built.
- Map can inform path planning or provide an intuitive visualization for a human or robot.

C. Cadena *et al.*, "Past, Present, and Future of Simultaneous Localization And Mapping: Towards the Robust-Perception Age," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1309–1332, 2016.

© Copyright Unmanned Systems Research Group

Questions

Do autonomous unmanned systems / robots really need SLAM?

- Yes, since many applications implicitly or explicitly do require a globally consistent map
- SLAM as a mechanism to compute a sufficient statistic that summarizes all past observations of the robot

Is SLAM completely solved?

- Not yet, SLAM is such a broad topic that the question is well posed only for a given robot / environment / performance combination
- Current SLAM algorithm can be easily induced to fail when either the motion of the robot or the environment are too challenging
  - ❖ Fast robot dynamics
  - ❖ Highly dynamic environments
  - ❖ Semantic fusion
  - ❖ Collaborative mapping



© Copyright Unmanned Systems Research Group

SLAM Classification

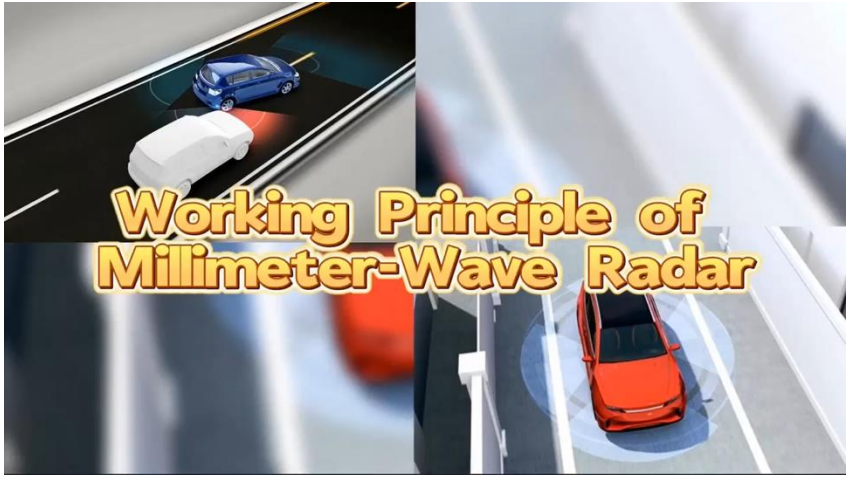
LiDAR SLAM:
- LiDAR + Radar
- LiDAR + IMU

Visual SLAM:
- Mono / Multi Camera (s)
- Camera + IMU = VIO-SLAM

Others:
LiDAR + Marker + IMU + Wheel = VGA (Visual-Gyroscopic-Aided)

LiDAR + Camera + IMU

SLAM is a multi-sensor fusion problem!

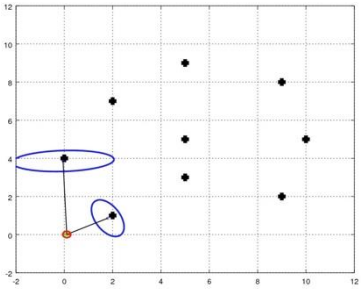


© Copyright Unmanned Systems Research Group

SLAM Classification

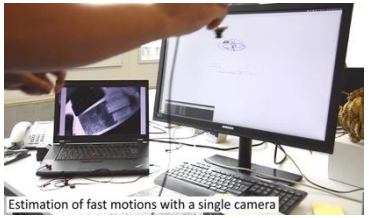
Bayesian Filtering based SLAM

- Theoretical fundamental and prototype of traditional Bayesian filtering based SLAM framework emerged in 1900s
- e.g., EKF SLAM, FastSLAM



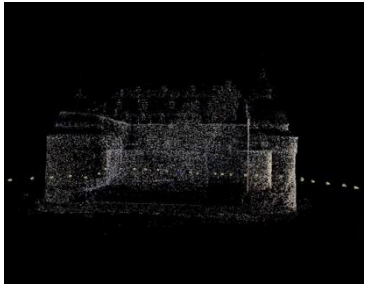
Visual odometry

- The process of estimating the ego-motion of a robot using only the input of a single or multiple cameras attached to it.
- e.g., stereo VO, monocular VO



Structure from motion

- Investigating the problem of recovering relative camera poses and 3D structure from a set of camera images
- Off-line version of visual SLAM



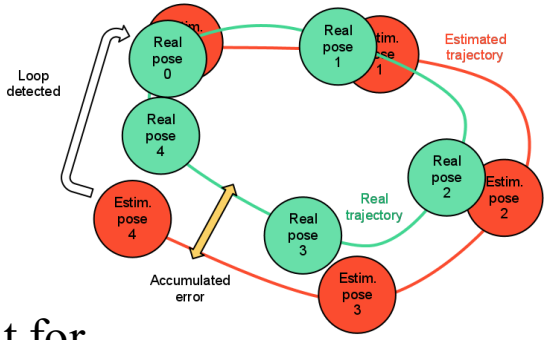
© Copyright Unmanned Systems Research Group

SLAM: large-scale mapping

The solution to large scale map management
- graph based slam and loop closure detection
- efficient map representation and refinement: sparse, dense, and semi-dense

Graph based SLAM

- constructing **a graph (位姿图)** whose nodes represent robot poses or landmarks
- **edge between nodes** encodes a sensor measurement that constrains connected poses



Loop closure detection (LCD)

- Detecting loop closures in a map to give additional constraint for the consistent mapping

© Copyright Unmanned Systems Research Group

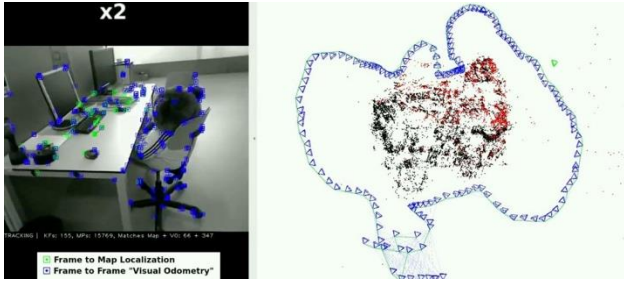
SLAM: State-of-the-Art (SOTA)

Sparse SLAM

- Only use a small selected subset of the pixels (features) from a monocular color camera
- Fast and real time on CPU but it produces a sparse map (point clouds)
- Landmark-based or feature-based representations

ORB (Oriented FAST and Rotated BRIEF) SLAM

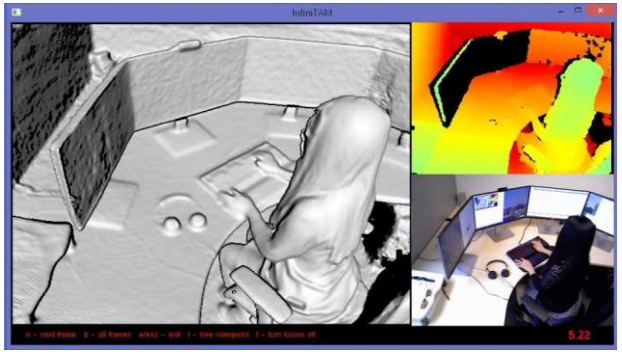
- One of the SOTA frameworks in the sparse SLAM category
- Complete SLAM system for monocular camera
- Real-time on standard CPUs in a wide variety of environments
  - ❖ Small hand-held indoors
  - ❖ UAV flying in industrial environments
  - ❖ Car driving around a city



© Copyright Unmanned Systems Research Group

Dense SLAM

- Use most or all of the pixels in each received frame
- Or use depth images from a depth camera
- It produces a dense map but GPU acceleration is necessary for the real-time operation.
- Volumetric model or surfel-based representations



InfiniTam

- One of the SOTA frameworks in the Dense SLAM category
- Multi-platform framework for real-time, large-scale depth fusion and tracking
- Densely reconstructed 3D scene

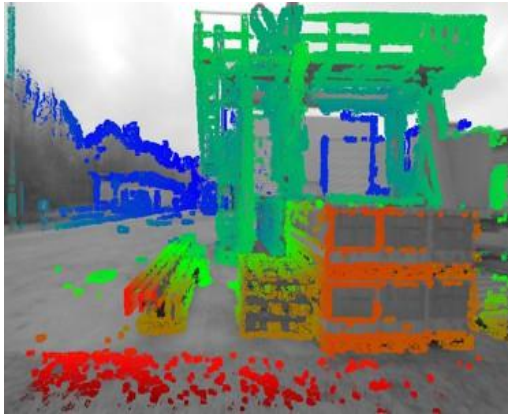
© Copyright Unmanned Systems Research Group

Direct method (semi-dense SLAM)

- Make use of pixel intensities directly
- Enable using all information in the image
- It produces a semi-dense map
- Higher accuracy and robustness in particular even in environments with little keypoints

LSD SLAM

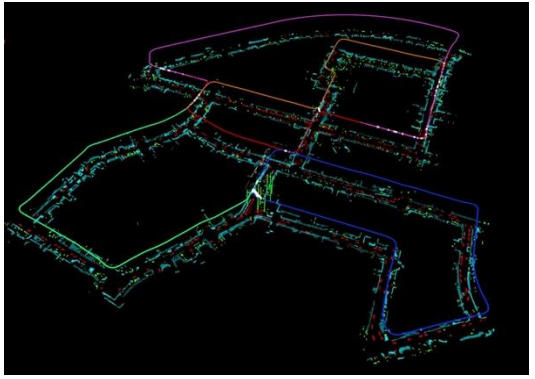
- Highly cited SLAM framework in the direct method SLAM category
- Large-scale, consistent maps of the environment
- Accurate pose estimation based on direct image alignment



© Copyright Unmanned Systems Research Group

- **Lidar SLAM**
  - Make use of the Lidar sensor input for the localization and mapping
  - Autonomous driving purpose-oriented in outdoor environment

- **Segmap**
  - Unified approach for Lidar SLAM based on the extraction of segments in 3D point clouds
  - Real-time single- and multi-agent systems

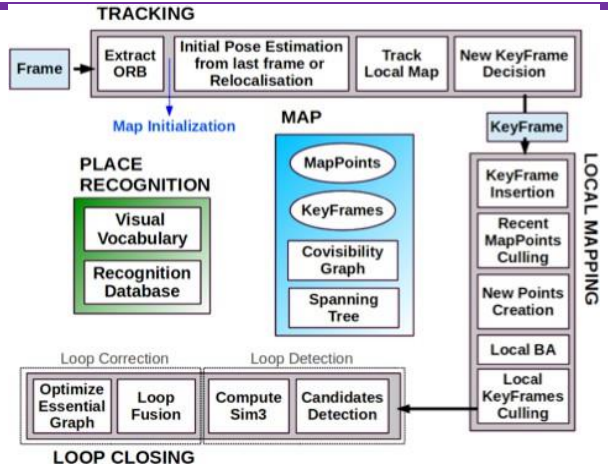


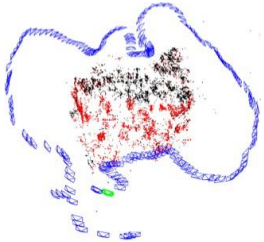
© Copyright Unmanned Systems Research Group

Example: ORB SLAM

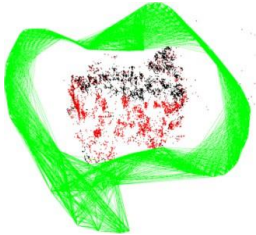
- Feature-based monocular SLAM system
  - Operates in real time, in small and large, indoor and outdoor environments

  https://github.com/raulmur/ORB_SLAM2
- Main components
  - Tracking
  - Local mapping
  - Loop closing
- Data structures
  - Map points
  - Keyframes
  - Covisibility graph
  - Essential graph





Map points & keyframes



Covisibility graph



Essential graph

R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1 163, 2015.

© Copyright Unmanned Systems Research Group

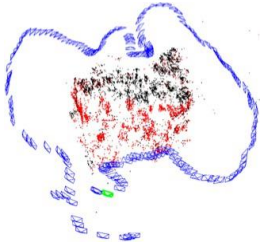
Data structures

Map points p_i

- 3D position $\mathbf{X}_{w,i}$
- Viewing direction $\mathbf{n}_i$
- Representative ORB descriptor $\mathbf{D}_i$

Keyframes K_i

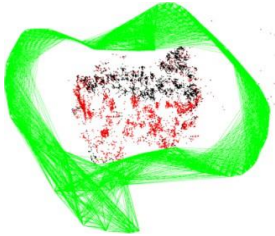
- Camera pose $\boldsymbol{T}_{iw}$
- Camera intrinsics (focal length and principal point)
- All the ORB features extracted in the frame



Map points & keyframes

Covisibility graph

- Undirected weighted graph
- Node: keyframe
- Edge: if two keyframes share observations of the same map points (**at least 15**)
- Weight θ: the number of common map points



Covisibility graph

Essential graph

- Retain all the nodes but less edges
- Subset of edges from the covisibility graph with high covisibility

Add loop closure edges



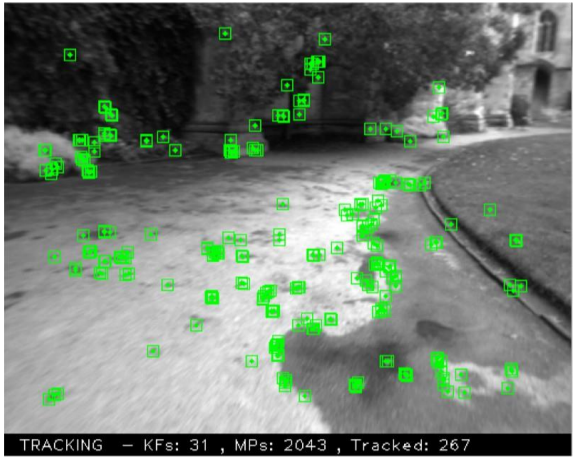
Essential graph

© Copyright Unmanned Systems Research Group

Tracking

ORB feature extraction

- For tracking, mapping, and place recognition tasks
- Robust to rotation and scale
- Good invariance to camera auto-gain and auto-exposure, and illumination changes
- Fast to extract and match allowing for real-time operation
- Show good precision/recall performances in **bag-of-word** place recognition



Example of ORB features

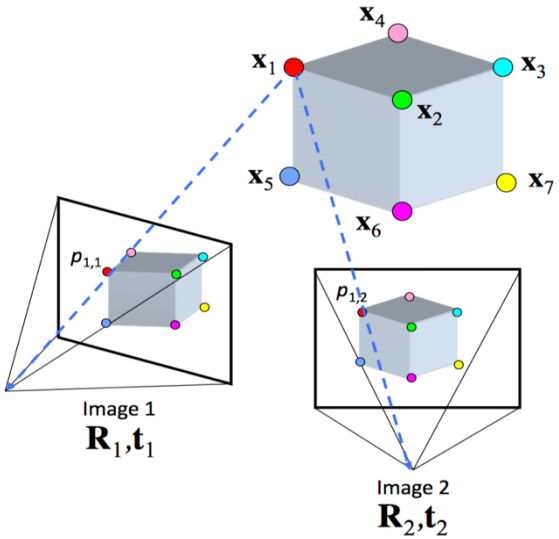


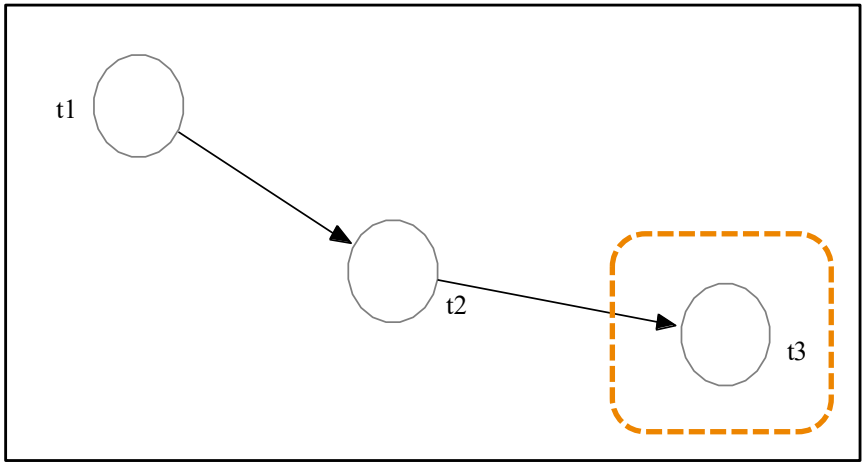
© Copyright Unmanned Systems Research Group

Tracking

Initial pose estimation

- Constant velocity motion model
- Bundle adjustment in two view case





$$E(\mathbf{P}, \mathbf{X}) = \sum_{i=1}^{m} \sum_{j=1}^{n} D\left(\mathbf{x}_{ij}, \mathbf{P}_i \mathbf{X}_j\right)^2$$

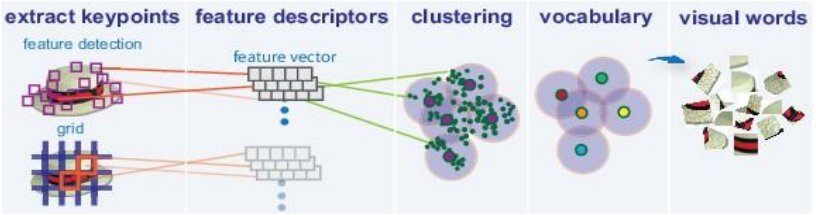
Minimizing the reprojection error

© Copyright Unmanned Systems Research Group

Bag-of-Visual-Words

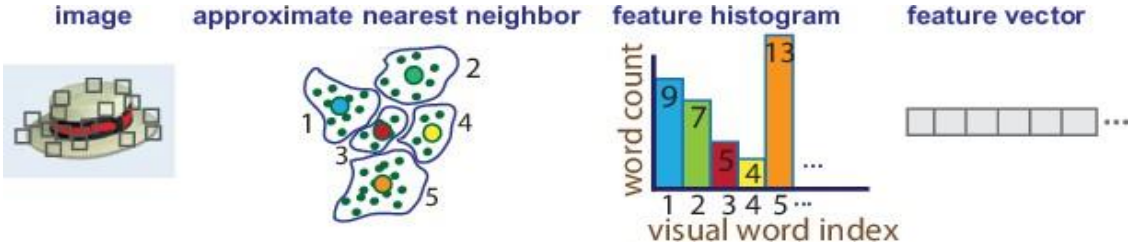
Training step

- Visual vocabulary is first created by clustering a large number of keypoint descriptors whose cluster centres from the visual words of the vocabulary.



Estimation step

- The local keypoints of a given image are first detected and described.
- Each descriptor is vector-quantized.
- The histogram of the vector-quantized keypoint descriptors is used as the image descriptor.

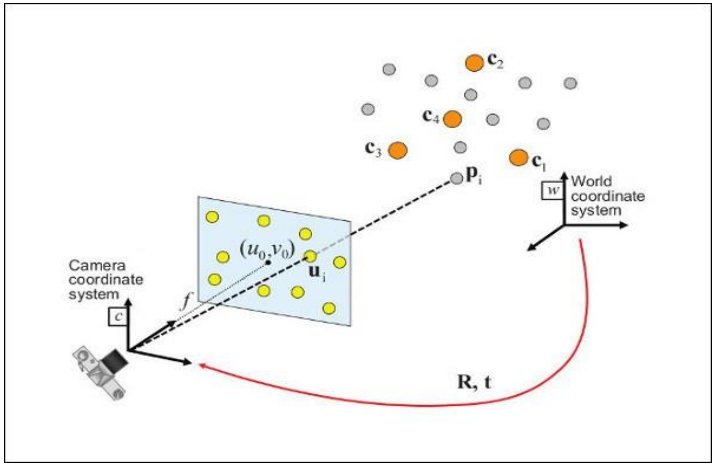


D. Gálvez-López and J. D. Tardós, "Bags of binary words for fast place recognition in image sequences," *IEEE Trans. Robot.*, vol. 28, no. 5, pp. 1188–1197, 2012.

© Copyright Unmanned Systems Research Group

Perspective-N-Point Problem

- Perspective-n-Point Problem
  - 2D-3D correspondences
  - To determine the position and orientation of a camera
    - Given its intrinsic parameters and
    - a set of n correspondences between 3D points and their 2D projections
  - Procedure
    - Estimate point clouds from 2D projections
    - Compute the transformation between the estimated point clouds and the given point clouds

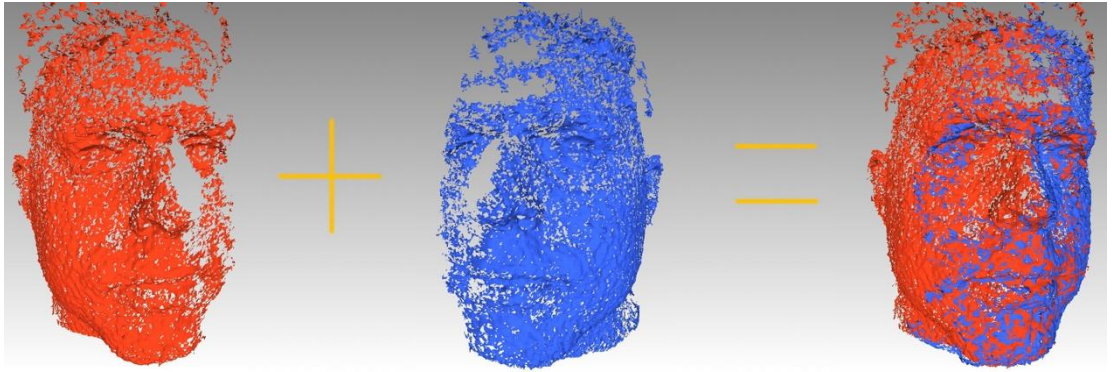


V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An accurate O(n) solution to the PnP problem," *Int. J. Comput. Vis.*, vol. 81, no. 2, pp. 155–166, 2009.

© Copyright Unmanned Systems Research Group

Iterative Closest Points (ICP)

- Widely used for geometric alignment of three-dimensional models
- Start with two meshes and an initial guess for their relative rigid-body transform
- Refine the transform by repeatedly generating pairs of corresponding points
- Related works
  - ❖ Point to Point
  - ❖ Point to Plane (tangent planes on the target surface)
  - ❖ …



© Copyright Unmanned Systems Research Group

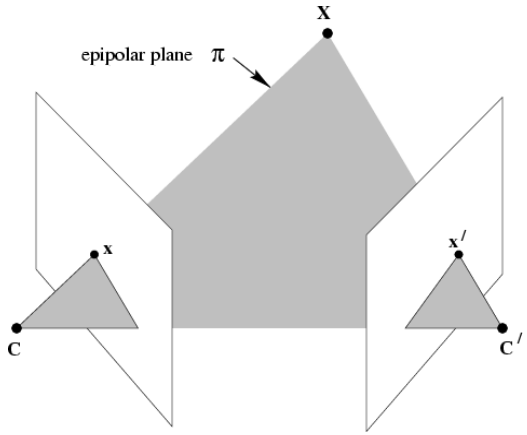
Local Mapping & Epipolar Geometry

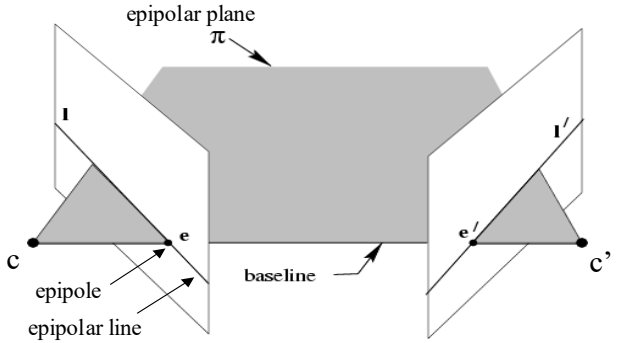
Keyframe insertion

- Compute the bags of words representation
- Update the covisibility graph

New map point creation

- Finding the matches by the epipolar geometry
- Initial map point creation via triangulation
- Consistency check
  - ❖ Parallax
  - ❖ Positive depth in both cameras
  - ❖ Reprojection error
  - ❖ Scale consistency check



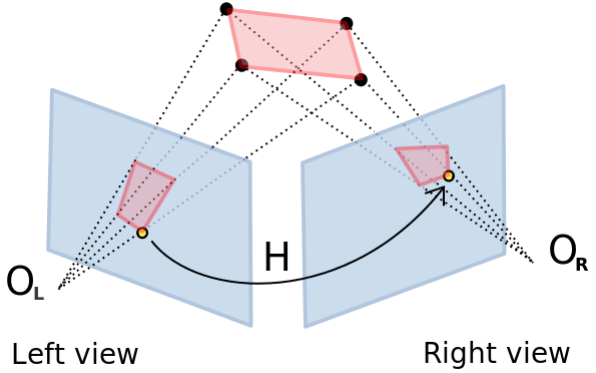


© Copyright Unmanned Systems Research Group

Homography

- Any two images of the same planar surface are related by a homography
- Properties of homography matrix
  - $\mathbf{x}' = \mathbf{H}\mathbf{x}$

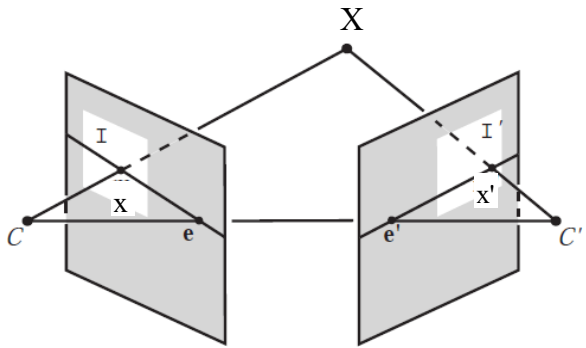
$$\begin{pmatrix} x'_1 \\ x'_2 \\ x'_3 \end{pmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$



© Copyright Unmanned Systems Research Group

Triangulation

- Generally, rays C→x and C’ → x’ will not exactly intersect
- Can solve via SVD, finding a least squares solution to a system of equations
- Procedure
  - Given camera projection matrix P, P’, and a correspondences x, x’
  - Cross product of x and PX should be zero. (self cross product)
  - Create matrix A
  - [U, S, V] = svd (A)
  - X = V(:, end)



© Copyright Unmanned Systems Research Group

Loop Closing

Loop fusion

- When the loop closure detection is triggered, the loop fusion is performed.
- Keyframe and map point correction
- Insert loop edges in the covisibility graph
- Current keyframe pose is corrected with the similarity transformation that we obtained
- Fuse duplicated map points

Essential graph optimization

- For all keyframes and all map points

© Copyright Unmanned Systems Research Group



Thank You!

© Copyright Unmanned Systems Research Group